

APPENDIX B:

Declaration of Original Work for SC2002 Assignment

We hereby declare that the attached group assignment has been researched, undertaken, completed, and submitted as a collective effort by the group members listed below.

We have honored the principles of academic integrity and have upheld Student Code of Academic Conduct in the completion of this work.

We understand that if plagiarism is found in the assignment, then lower marks or no marks will be awarded for the assessed work. In addition, disciplinary actions may be Taken.

Name	Matric No	Course/Lab Group	Signature/Date
Agnes Toh Li Wei	U2410481A	BACF/SACD	 19/11/25
Finn Wallace	N2504002G	EEE/SACD	19/11/25 
Cai YiJia	U2410492A	BACF/SACD	 19/11/25
Leong Shu Ting	U2410823E	BACF/SACD	 19/11/25
Ong Zi Yan	U2410710D	BACF/SACD	 19/11/25
Xie Leo	U2420185L	CSEC/SACD	 19/11/25

Chapter 1: Requirement Analysis & Features

1.1 Understanding the Problem and Requirements

Our group began by reading through the Internship Placement Management System document line-by-line and observed the capabilities and functionalities for different users. Based on the objective of this project, we are required to build an internship placement management system for 3 distinct user roles - Students, Career Centered Staff and Company Representative.

The key requirement outlines are:

- **User Authentication:** Users have to log in with their respective student ID(for students)/ Company email(company representatives ID)/NTU account(career center staff's ID).
- **Role-based access control:** Includes three distinct user roles, each with different permissions.
- **Internship opportunity Management:** Create, edit(before approval), delete, control visibility, Maintain status.
- **Application Management & Placement workflow:** Students can apply only to internships for which they are eligible.
- **Withdrawal Management:** Students may request withdrawal before or after accepting an offer.
- **Reporting and Filtering:** Career Center Staff will be able to generate reports filtered by status, preferred major, internship level, company, closing date.
- **File-Based Data Persistence:** User list loaded from provided student/staff list files during initialisation.

Implicit expectation and assumptions:

- Clear CLI menus and navigation per user role
- Consistent validation and meaningful error handling
- Data integrity rules (max 3 student applications, max 5 postings per rep)
- Visibility toggle does not affect a student's view of their own applications

Ambiguities Interpreted:

- Closing-date behaviour: treated as inclusive
- Students can view all historical applications
- Filters persist during the session but reset on logout
- Multiple company representatives per company supported

Chapter 2: Overall Architecture

2.1 Overall Architecture

This chapter explains how we translated our functional requirements into a clear, object-oriented architecture. Our system follows an Object Oriented , modular, and layered design, structured using the Boundary-Control-Entity(BCE) pattern:

- **Boundary Layer:**
 - Serving as the external interface, it provides a Command-Line Interface (CLI) tailored to each profile (Students, Career Centered Staff and Company Representative). Based on the menu option selected, the boundary component validates user inputs and maps them to the appropriate control methods, effectively delegating the request to the next layer for processing.
- **Control Layer:**
 - The control layer acts as the intermediary between the interface and the data model. After receiving validated input from the boundary layer, it interprets the user's request and coordinates the sequence of operations required to fulfil it. It also encapsulates key processes such as authentication, internship approval, application handling, filtering, and password-reset requests, ensuring that these workflows remain hidden behind well-defined control methods.
- **Entity Layer:**
 - The entity layer contains the core business objects that represent the systems's persistent data, such as **Users, Student, Company Representatives, Career Center Staff, Internship Opportunities, Applications, and Withdrawal Requests** and encapsulates its related attributes and behaviours. All business state changes, such as updating application status, incrementing confirmed slots, or modifying user profiles, occur exclusively through entity methods. This ensures data integrity, reusability, and high cohesion across the system.

2.2. Implementation

Tools used:

- Java 17
- IDE: IntelliJ / Eclipse
- Version control: GitHub
- Class & Use Case Diagram: Visual Paradigm

Sample code Snippets

- Encapsulation

```
public class User implements Serializable{
    private String userID;
    private String name;
    private String password;
    private String email;

    public User(String userID, String name, String email) {
        this.userID = userID;
        this.name = name;
        this.email = email;
        this.password = "password";
    }

    public boolean login(String passwordAttempt) {
        return this.password.equals(passwordAttempt);
    }
}
```

As seen above, the password attribute is declared as a private field, preventing direct access from outside the class. The login(String passwordAttempt) method provides controlled access by allowing the system to verify the password without exposing its actual value, demonstrating encapsulation.

- Inheritance

```
public class CompanyRepresentative extends User implements Serializable {

    private static int nextId = 1001;
    private String companyName;
    private String department;
    private String position;
    private String accountStatus;
    private List<InternshipOpportunity> createdOpportunities;
    private FilterSettings filterSettings;

    public CompanyRepresentative( String name,String email,
        String companyName, String department, String position) {
        super("", name, email);
        this.companyName = companyName;
        this.department = department;
        this.position = position;
        this.accountStatus = "Pending";
        this.createdOpportunities = new ArrayList<>();
        this.filterSettings = new FilterSettings();
    }
}
```

As seen above, it reflects inheritance as the Company inherits from User.

- Error handling

```
try (BufferedReader reader = new BufferedReader(new InputStreamReader(is))) {
    String line;
    while ((line = reader.readLine()) != null) {
        String[] parts = line.split(",");
        if (parts.length >= 4) {
            String userId = parts[0].trim();
            String name = parts[1].trim();
            String major = parts[2].trim();
            int yearOfStudy = Integer.parseInt(parts[3].trim());
            String email = parts[4].trim();

            Student student = new Student(userId, name, email, yearOfStudy, major);
            state.students.put(email, student);
        }
    }
} catch (IOException e) {
    System.out.println("Error reading " + STUDENT_DATA_FILE + ". Starting with empty student list.");
    addSampleStudents();
} catch (NumberFormatException e) {
    System.out.println("Error parsing number in " + STUDENT_DATA_FILE + ". Starting with empty student list.");
    addSampleStudents();
}
```

Above reflects the error handling when it comes to loading student data from files. It prints informative messages when errors are caught.

- Interface

```
public class Application implements Serializable {
    private static int nextApplicationId = 1000;

    private String applicationId;
    private Student applicant;
    private InternshipOpportunity opportunity;
    private ApplicationStatus status;
    private LocalDate applicationDate;
    private LocalDate statusUpdateDate;
    private String rejectionReason;
    private String withdrawalReason;
    private boolean placementAccepted;
}
```

Figure shows that Application implements the Serializable interface.

2.3 Application of OOD Principles (SOLID)

Single Responsibility Principles (SRP): Every class in the system has a single, focused purpose - a single reason to change. For instance, the **Application** manages internship applications, **SystemState** handles data loading/saving, and user subclasses manage role-specific logic. This makes our code modular and easier to test.

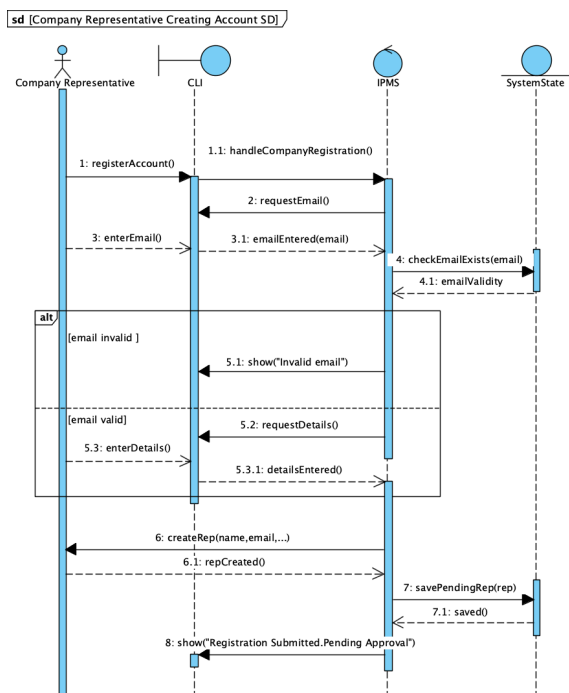
Open/Closed Principle (OCP): Our design supports extension without modifying existing code. Adding a new internship level or user role only requires extending enums or base classes, without altering core logic.

1. Creating Internship Opportunity
2. View and Edit Internship Opportunity
3. View and Manage Applications
4. View All Internship Opportunities

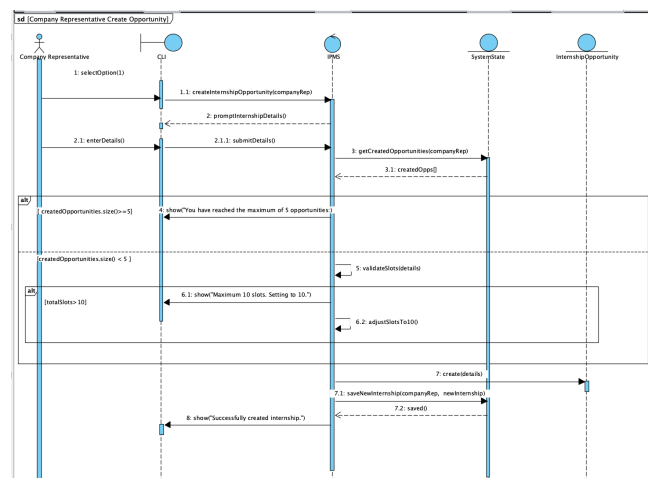
For each option, the CLI acts as a boundary object that handles user input/outputs such as prompts, menus, or messages. It forwards the request to IPMS, the control object that encapsulates the core business logic (e.g. retrieving created opportunities or validating the number of slots).

IPMS then coordinates with the entity objects such as SystemState, InternshipOpportunity, or Application to fetch/update data, while the diagrams use loop and alt fragments to show conditional behaviour (e.g. max 5 opportunities, only approved internships can be toggled).

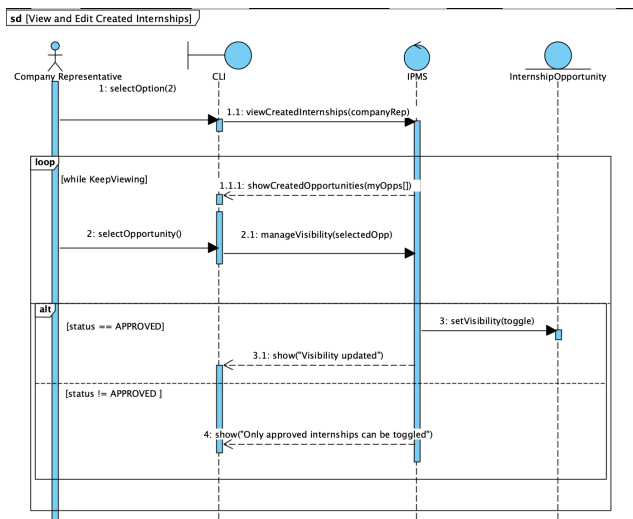
Creating a Company Rep Account



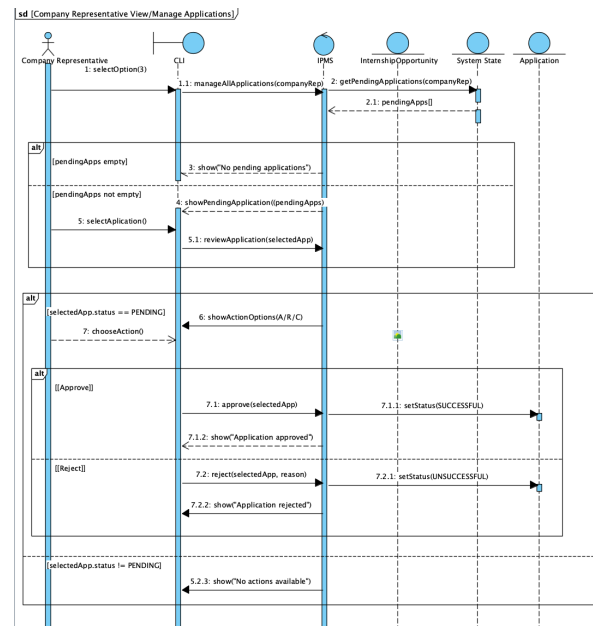
Use Case 1: Creating Internship Opportunity



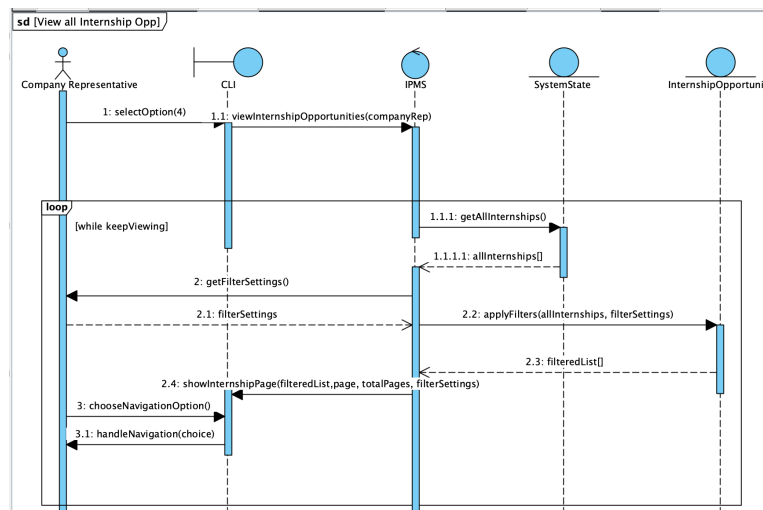
Use Case 2: View and Edit Created Internships



Use Case 3: View and Manage Applications



Use Case 4: View all Internship Opportunities



5. Additional Features/Functionalities

We added additional features to enhance user experience. When a user forgets their password during login, they can select a “Forgot Password” option. The system records a password-reset request, which will be directed to Career Center Staff for verification and approval.

Upon approval, the system will automatically resets the user's password to the default ("password"), and the user is required to change it upon their next login. This improves security and workflow by preventing unauthorised resets, and adds a realistic administrative process that aligns with the role-based permissions of the system.

We also added a 'Clear the System' function. This feature allows the system administrator to wipe the entire database. It is particularly useful for testing, resetting the system between demonstrations, or preparing for a new academic cycle.

6. Testing

S	ID	Test Case	Expected Outcome	Failure Condition	Check
1	1	Valid User Login	User can log in and access their dashboard according to their role.	User cannot log in or receives incorrect error messages.	✓
	2	Invalid ID	System notifies the user of incorrect ID.	User can log in with an invalid ID or receives no meaningful error message.	✓
	3	Incorrect Password	System denies access and alerts the user of incorrect password.	User logs in successfully with a wrong password or receives no meaningful error message.	✓
	4	Password Change Functionality	System updates password, prompts re-login, and allows login with new credentials.	System does not update the password or denies access with the new password.	✓
	5	Company Representative Account Creation	Representative can only log in after approval by Career Center Staff.	Representative can log in without authorization.	✓
2	10	Company Representative Opportunity Creation	System allows creation only when requirements are met.	System allows invalid data or exceeding allowed number of opportunities.	✓

	16	Create, Edit, Delete Internship Opportunities	Representatives can add, edit (before approval), and delete opportunities.	System cannot perform these actions or errors occur.	✓
	12	Internship Detail Access for Company Representative	Representatives can always access full details of their own opportunities.	Details become inaccessible when visibility is toggled off.	✓
	18	Toggle Internship Opportunity Visibility	Visibility changes correctly affect what students see.	Visibility settings do not update or do not affect listings.	✓
3	17	Career Center Staff Internship Opportunity Approval	Staff can review, approve/reject, and approved listings become visible to students.	Staff cannot review, approval/rejection fails, or visibility does not update.	✓
	11	Internship Opportunity Approval Status	Representatives can view pending/approved/rejected statuses correctly.	Status updates are missing, incorrect, or not saved.	✓
	13	Restriction on Editing Approved Opportunities	Editing is disabled after approval by Career Center Staff.	Representatives can edit after approval.	✓
4	6	Internship Opportunity Visibility (Student View)	Students see opportunities relevant to their year, major, internship level, and visibility settings.	Students see irrelevant opportunities or see them when visibility is off.	✓
	7	Internship Application Eligibility	Students can apply only for opportunities matching their profile and only when visibility is on.	Students apply for irrelevant opportunities or apply when visibility is off.	✓
	8	Viewing Application Status After Visibility Off	Students continue to view previously submitted applications	Application details become inaccessible after visibility is toggled off.	✓

			even when visibility is off.		
	9	Single Internship Placement Acceptance	System allows one accepted placement and auto-withdraws all others.	Student accepts multiple placements or other applications remain active.	✓
5	14	Student Application Management & Placement Confirmation	Representatives retrieve correct applications, update slots, and confirm placements accurately.	Incorrect application retrieval, incorrect slot updating, or confirmation not reflected.	✓
	15	Placement Confirmation Status Update	Status updates accurately reflect confirmation.	Status fails to update or updates incorrectly.	✓
6	19	Career Center Staff Internship Opportunity Management	Withdrawal approvals and rejections are processed correctly, with system updates to reflect the decision and slot availability changes	Incorrect or failed processing of withdrawal requests or slot counts not updating properly	✓
7	20	Generate and Filter Internship Opportunities	Accurate report generation with options to filter by placement status, major, company, level, and other specified categories	Reports are inaccurate, incomplete, or filtering does not work as expected	✓

Reflections & challenges

What went well?

We have smoothly implemented the workflow logic. Key workflows such as application approval, placement confirmation, visibility toggling, and withdrawal management were implemented without major rework. This showed that our initial design decisions were sound. Additionally, we managed to implement extra features such as the forgot-password workflow with staff approval.

Lessons learned from OODP

- Code reusability

- User superclass: Student, Company Representative, and Career Center Staff inherited shared attributes and methods (ID, name, password, login logic). This reduced duplication and ensured consistent behaviour across all user roles.
- Enums: InternshipStatus, ApplicationStatus, and InternshipLevel centralised all valid states, making the code cleaner, easier to maintain, and less error-prone.
- Controllers: Common operations were abstracted into controllers (e.g., InternshipController, ApplicationController), allowing multiple boundary classes to reuse the same business logic without rewriting functions.
- Purpose of Class and Sequence Diagram
 - Class diagram helps us to visualise the overall system structure, class responsibilities, and identify where inheritance or polymorphism should be used.
 - Sequence diagram helps us to structure the entire process flow and identify the interaction of each class. This prevents logical gaps and reduces design errors during implementation.

What could be improved?

1. Time management

With the large requirements of the project and differences in our schedule with the group members, we could only communicate things through telegram and complete the entire project at our own time. In the future, we will set ourselves timeline.

2. Code implementation

Moreover, implementing the rule-heavy logic (such as limiting applications based on student year and major, auto-withdrawing applications upon acceptance, and managing approval workflows) required extensive testing and debugging to get all the conditions right. The team also had to coordinate effectively using version control; merging code from 3 different teammates which leads to integration debug.

GitHub Repositor

<https://github.com/Finnthefrog/SC2002Project-InternshipPlacementManagementSystem>